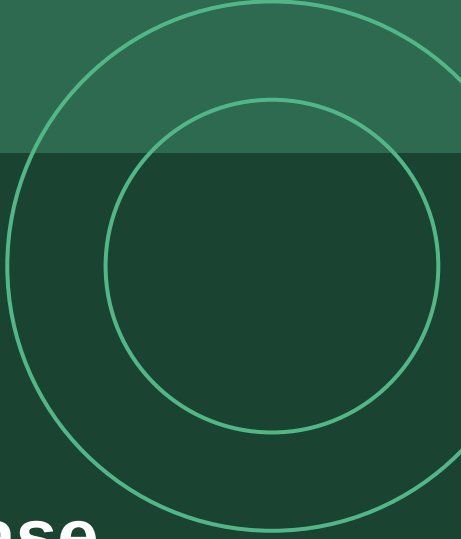




ProtoPNet Crop Disease

Research Pipeline Documentation

Stage 2b — Data Augmentation Pipeline v2

Target-Count Mode | Dual Pipeline | PlantWild v2

1. Overview

What v2 does differently and why it matters

Augmentation is Stage 2b of the ProtoPNet pipeline, running after preprocessing and class cleanup. Its purpose is to expand the training set by generating synthetic variations of existing images, giving the model more examples to learn from without collecting new data.

Version 2 of this script introduces Target-Count Mode — a fundamentally different approach to augmentation that directly solves the severe class imbalance identified during preprocessing. Instead of generating a fixed number of copies per image, v2 brings every class to exactly the same total image count. This means majority classes may have excess files deleted, while minority classes receive as many augmented copies as needed to reach the target.

Note: Val and test sets are NEVER touched by this script. Augmentation is applied exclusively to the train/ split. This preserves the integrity of evaluation.

Key differences: v1 vs v2

Feature	v1 (original)	v2 (your modified version)
Mode	Fixed copies per image	Target count per class
Minority classes	Boost multiplier (7x)	Generate exactly TARGET - current
Majority classes	Same copies as others	Delete excess augmented files
Pipeline	Single standard pipeline	Standard + Strong (auto-selected)
Strong aug	Not available	Activates when gap/originals > 5x
Extra transforms	None	Elastic distortion + Random Erasing
Re-run safety	May double-augment	Detects existing aug files, no duplicates
Class balance	Partial (boost helps)	Perfect — every class = TARGET images

Pipeline at a Glance

Step	Function	What it does
1	scan_train_dir()	Scans train/ — separates originals from existing aug_files

Step	Function	What it does
2	build_pipelines()	Builds standard and strong torchvision transform pipelines
3	run_target_count_mode()	Core logic — trims over-target classes, generates for under-target
4	compute_copies_per_class()	Legacy fallback — only used if TARGET_IMAGES_PER_CLASS = None
5	copy_originals_if_needed()	Copies originals to OUTPUT_DIR if not using 'same' mode
6	write_summary()	Writes augmentation_summary.txt with per-class stats

2. Target-Count Mode

The core v2 logic — how every class reaches TARGET_IMAGES_PER_CLASS

This is the most important change in v2. The setting TARGET_IMAGES_PER_CLASS (default: 1000) defines the exact number of images every class should have in the training set after augmentation. The script calculates the gap for each class and takes one of three actions.

Three possible outcomes per class:

A

Over target — TRIM

If a class already has more than TARGET images total (originals + existing augmented), excess augmented files are deleted. The oldest aug_ files are removed first (sorted by name). Originals are NEVER deleted regardless of how many there are.

B

At target — SKIP

If a class already has exactly TARGET images, nothing happens. This makes the script safe to re-run — it will not generate duplicates or touch classes that are already balanced.

C

Under target — GENERATE

If a class has fewer than TARGET images, the script generates exactly (TARGET - current) new augmented images. It cycles through the original images in shuffled order, re-shuffling after every full cycle to maximise visual variety across the generated set.

The gap ratio and pipeline selection:

When generating for an under-target class, the script computes a gap ratio:

$$\text{gap_ratio} = (\text{TARGET} - \text{current_total}) / \text{number_of_originals}$$

This tells us how many copies per original are needed. If gap_ratio exceeds STRONG_AUG_MULTIPLIER_THRESHOLD (default: 5), the strong augmentation pipeline is used instead of the standard one. The reasoning: if you need to generate 50 copies from 10 originals, applying only mild augmentation will produce near-identical images that do not add useful diversity. The strong pipeline introduces more aggressive transformations to maximise visual variation.

gap_ratio	Pipeline selected	Example scenario
<= 5x	Standard	Class has 200 originals, needs 1000 total — gap=800, ratio=4x
> 5x	STRONG	Class has 30 originals, needs 1000 total — gap=970, ratio=32x

Note: For PlantWild with TARGET=1000, most classes with 100+ originals will use the standard pipeline. The tiny classes (under ~50 originals) will trigger the strong pipeline automatically — exactly the classes that need maximum diversity.

Cycling through originals:

When a class needs more augmented images than it has originals, the script cycles through the originals repeatedly. After every full cycle, the order is re-shuffled. This ensures no single original image is the exclusive source of many consecutive augmented copies, which would reduce effective diversity in the training set.

```
# Cycle logic (simplified from run_target_count_mode)
shuffled_originals = random.shuffle(originals.copy())
while n_generated < n_needed:
    src = shuffled_originals[n_generated % len(shuffled_originals)]
    if n_generated > 0 and n_generated % len(originals) == 0:
        random.shuffle(shuffled_originals)    # re-shuffle each full cycle
    aug_img = pipeline(src)
    save(aug_img)
```

3. Augmentation Pipelines

Standard and Strong — what each transform does and why

The script builds two torchvision Compose pipelines. Both are constructed in `build_pipelines()` from CONFIG settings. The standard pipeline matches the ProtoPNet research workflow specification exactly. The strong pipeline extends it with more aggressive parameters plus two additional transforms: random erasing and wider crop/rotation ranges.

Important: neither pipeline applies `ToTensor()` or `Normalize()` (except momentarily inside the strong pipeline for `RandomErasing`, which requires tensor format, then converts back to PIL). The saved augmented files are standard JPEG images. Normalisation happens in the PyTorch DataLoader transform at training time — not here.

Transform-by-transform breakdown:

	Standard	Strong	Purpose
	scale=0.7-1.0 ratio=0.75-1.333 LANCZOS	scale=0.5-1.0 ratio=0.65-1.50 LANCZOS	Forces model to focus on disease lesion regions rather than memorising full-frame composition. Strong mode uses wider scale (0.5) to simulate extreme close-ups.
	Standard	Strong	Purpose
	p=0.5	p=0.5 (same)	Plant disease lesions appear on both sides of leaves. Horizontal mirroring doubles effective viewpoint diversity at zero cost.
	Standard	Strong	Purpose
	p=0.3	p=0.3 (same)	Field photographs are sometimes taken upside-down. Lower probability than horizontal (0.3 vs 0.5) because most photos have natural top/bottom orientation.
	Standard	Strong	Purpose
	degrees=+-30 fill=black	degrees=+-45 fill=black	Leaf angle varies significantly in field conditions. Strong mode extends to +-45 degrees for more aggressive viewpoint variation. Black fill is acceptable — disease classifiers learn to ignore uniform background padding.

	Standard	Strong	Purpose
	brightness=0.3 contrast=0.3 sat=0.2 hue=0.05	brightness=0.5 contrast=0.5 sat=0.4 hue=0.1	Critical for PlantWild which has large lighting variance (in-the-wild conditions: shade, direct sun, phone cameras). Strong mode doubles the ranges to simulate extreme lighting.
	Standard	Strong	Purpose
	p=0.2 kernel=5 sigma=0.1-2.0	p=0.4 kernel=5 sigma=0.1-2.5	Simulates natural image quality variation in field photography. Strong mode applies more frequently and with slightly wider sigma range.
	Standard	Strong	Purpose
	Not applied	p=0.3 scale=0.02-0.20 ratio=0.3-3.3	Simulates occluded disease patches — leaves partially covered by other leaves, mud, insects. Prevents the model from relying on any single image region. Requires ToTensor conversion internally, then converts back to PIL.

Pipeline execution order (both pipelines):

```

Input image (224x224 PIL RGB)
|
v 1. RandomResizedCrop    -> crops sub-region, resizes to 224x224
|
v 2. RandomHorizontalFlip -> 50% chance mirror
|
v 3. RandomVerticalFlip   -> 30% chance flip
|
v 4. RandomRotation       -> random angle within range
|
v 5. ColorJitter          -> random brightness/contrast/saturation/hue
|
v 6. GaussianBlur         -> 20% or 40% chance blur
|
v 7. RandomErasing        -> STRONG only: 30% chance black patch
|
Output image (224x224 PIL RGB) -> saved as JPEG at quality=95

```

4. Step 1 — Scanning the Training Directory

How the script reads and categorises existing files

Before any augmentation can happen, the script must understand the current state of the training directory — how many original images each class has, and how many augmented copies already exist from a previous run.

The `scan_train_dir()` function walks every class subfolder in `TRAIN_DIR` and separates files into two categories based on the `AUG_PREFIX` setting (default: 'aug'):

- **Originals** — any image file whose stem does NOT start with 'aug_'. These are the preprocessed images from `preprocess.py`. They are never deleted by this script.
- **Augmented** — any image file whose stem starts with 'aug_'. These were created by a previous run of this script. They may be deleted if the class is over-target.

```
# Output structure of scan_train_dir()
{
  "tomato early blight": {
    "originals": [Path("img001.jpg"), Path("img002.jpg"), ...], # never touched
    "augmented": [Path("aug_img001_00001.jpg"), ...]           # may be deleted
  },
  "coffee black rot": {
    "originals": [Path("img006.jpg"), Path("img007.jpg")],      # only 2 original
    "augmented": []                                             # no previous au
  },
  ...
}
```

Note: The separation of originals from augmented files is what makes the script safe to re-run. Running it twice will not double-augment your dataset — it will detect existing `aug_` files and account for them when calculating how many more to generate.

Augmented filename format:

```
# Pattern: aug_<original_stem>_<5-digit-index>.jpg
aug_img001_00001.jpg  # first augmented copy of img001.jpg
aug_img001_00002.jpg  # second augmented copy of img001.jpg
aug_img002_00003.jpg  # first augmented copy of img002.jpg

# The 5-digit index is global per class — not per original.
# This avoids filename collisions when cycling through originals.
```


5. Configuration Reference

Every setting explained

Setting	Default	Description
TRAIN_DIR	./Dataset/plantwild/train	Path to train/ folder from preprocess.py
OUTPUT_MODE	same	'same' saves aug files alongside originals. Any path saves to a new folder.
OUTPUT_DIR	./Dataset/plantwild_augmented/train	Only used when OUTPUT_MODE is not 'same'
TARGET_IMAGES_PER_CLASS	1000	Target count per class. Set None to use legacy mode.
STRONG_AUG_MULTIPLIER_THRESHOLD	5	gap/originals ratio above which strong pipeline activates
COPIES_PER_IMAGE	3	Legacy mode only: copies per original image
ENABLE_BOOST	True	Legacy mode only: enable minority class boost
MIN_SAMPLES_FOR_BOOST	50	Legacy mode only: boost threshold
BOOST_COPIES_PER_IMAGE	7	Legacy mode only: copies for boosted classes
FLIP_H_PROB	0.5	Horizontal flip probability
FLIP_V_PROB	0.3	Vertical flip probability
ROTATION_DEGREES	30	Standard rotation range +-degrees
COLOR_BRIGHTNESS	0.3	Standard colour jitter brightness factor
COLOR_CONTRAST	0.3	Standard colour jitter contrast factor
COLOR_SATURATION	0.2	Standard colour jitter saturation factor
COLOR_HUE	0.05	Standard colour jitter hue factor
CROP_SCALE_MIN	0.7	Standard minimum crop scale
BLUR_PROB	0.2	Standard Gaussian blur probability
BLUR_KERNEL_MAX	5	Gaussian blur kernel size (must be odd)

Setting	Default	Description
STRONG_ROTATION_DEGREES	45	Strong pipeline rotation range
STRONG_COLOR_BRIGHTNESS	0.5	Strong colour jitter brightness
STRONG_COLOR_CONTRAST	0.5	Strong colour jitter contrast
STRONG_COLOR_SATURATION	0.4	Strong colour jitter saturation
STRONG_COLOR_HUE	0.1	Strong colour jitter hue
STRONG_CROP_SCALE_MIN	0.5	Strong minimum crop scale
STRONG_BLUR_PROB	0.4	Strong Gaussian blur probability
STRONG_ERASE_PROB	0.3	Strong random erasing probability
OUTPUT_IMG_SIZE	(224, 224)	Output image dimensions — must match backbone
SAVE_FORMAT	JPEG	Output format: JPEG or PNG
SAVE_QUALITY	95	JPEG save quality
AUG_PREFIX	aug	Prefix used to identify augmented files
RANDOM_SEED	42	Seed for reproducibility
LOG_FILE	augmentation.log	Path for full run log

6. Legacy Mode

Backward-compatible fallback when `TARGET_IMAGES_PER_CLASS = None`

If `TARGET_IMAGES_PER_CLASS` is set to `None`, the script falls back to the v1 behaviour: generate a fixed number of copies per original image. Minority classes can receive a higher copy count via the boost settings.

```
# To use legacy mode, set in CONFIG:
"TARGET_IMAGES_PER_CLASS": None,      # disables target-count mode
"COPIES_PER_IMAGE": 3,                # every image gets 3 copies
"ENABLE_BOOST": True,                 # minority classes get more
"MIN_SAMPLES_FOR_BOOST": 50,          # below this = minority
"BOOST_COPIES_PER_IMAGE": 7,          # minority classes get 7 copies
```

Important: Legacy mode does NOT balance classes — it only reduces the imbalance ratio. Target-count mode is strongly recommended for ProtoPNet because class imbalance directly harms prototype quality for minority classes.

7. Output & Summary Report

What the script produces on disk

When `OUTPUT_MODE` is 'same' (recommended), augmented files are written directly into the existing class folders alongside originals. PyTorch's `ImageFolder` loads both automatically with no extra configuration.

```
plantwild/train/
  tomato early blight/
    img001.jpg          <- original (never touched)
    img002.jpg          <- original
    aug_img001_00001.jpg <- generated
    aug_img001_00002.jpg <- generated
    aug_img002_00003.jpg <- generated
    ...                 <- up to TARGET total files
  coffee black rot/
    img006.jpg          <- original
    aug_img006_00001.jpg <- generated x many to reach TARGET
    ...
```

augmentation_summary.txt — example output:

===== AUGMENTATION SUMMARY =====

Train directory : ./Dataset/plantwild/train
 Output mode : same
 Mode : Target-count (1000/class)

Total classes : 111
 Original images : 6,590
 Augmented images deleted : 0
 Augmented images created : 104,410
 Final train total : 111,000
 Effective multiplier : 16.84x

Class	Orig	Del	Gen	Total	Status
tomato early blight	213	0	787	1000	topped up (standard)
citrus canker	219	0	781	1000	topped up (standard)
coffee black rot	4	0	996	1000	topped up (STRONG)
...					

=====

8. Imbalance & Weighted Loss

What augmentation fixes and what still needs attention at training time

Target-count augmentation brings every class to the same total count in the training set. This eliminates frequency-based imbalance — the model sees each class equally often per epoch. However, there is a subtlety worth understanding for your paper.

What target-count augmentation fixes:

- Training frequency imbalance — every class seen equally per epoch
- Prototype push quality — more images means better candidate patches to select from
- Gradient domination by majority classes — loss is now balanced across classes

What it does NOT fully fix:

- Diversity imbalance — a class with 4 originals boosted to 1000 still only has 4 unique source images. All 996 augmented copies are transformations of those 4. The strong pipeline mitigates this but cannot eliminate it.
- Test set imbalance — your test set still reflects the original distribution (1 test image for tiny classes). Evaluation metrics for those classes remain unreliable.

Note: This is why the class cleanup script (removing classes with < 10 train images) should run BEFORE augmentation. Augmenting 4 images to 1000 is technically possible but scientifically questionable. Removing them is the cleaner research decision.

Weighted loss — still recommended:

Even with balanced training counts, it is good practice to use weighted cross-entropy during training. After augmentation the counts are equal, so weights will all be approximately 1.0 — but the code is robust to future CONFIG changes and documents your intent clearly.

```
# In your training script – compute after building the dataset
train_dataset = datasets.ImageFolder('./Dataset/plantwild/train')
class_counts = [0] * len(train_dataset.classes)
for _, label in train_dataset.samples:
    class_counts[label] += 1

weights = [1.0 / c for c in class_counts]
w_tensor = torch.FloatTensor(weights).to(device)
criterion = torch.nn.CrossEntropyLoss(weight=w_tensor)
```

9. Full Pipeline Order & Next Steps

1

preprocess.py — DONE

Raw images cleaned, deduplicated, blur-filtered, split 70/15/15. Output: plantwild/train|val|test/

2

cleanup_classes.py — DONE

Classes with fewer than 10 train images removed from all three splits.

3

augment-2.py — YOU ARE HERE

Run this script. Every class in train/ will be brought to TARGET_IMAGES_PER_CLASS (default 1000). Val and test are untouched.

4

Verify augmentation output

Check augmentation_summary.txt. Confirm all classes show total=1000 (or TARGET). Spot-check a few aug_ files visually to confirm transforms look correct.

5

Stage 3 — ProtoPNet Architecture & Training

Build the DataLoader pointing at plantwild/train/ with ImageNet normalisation. Set up DenseNet161 backbone, add-on layers, prototype layer (111 classes x 10 = 1110 prototypes). Begin Phase 1 warm-up training.